
schedulerlib

Release 0

La Sapienza, Università di Roma, Emiliano Casalicchio, Antonino

May 06, 2026

CONTENTS:

Add your content using reStructuredText syntax. See the [reStructuredText](#) documentation for details.

API REFERENCE

Python library to create satellites and tasks and solve the scheduling problem with a genetic programming algorithm.

class scheduler.ExprNode

Bases: pybind11_object

A node in the expression tree representing an individual in the Genetic Programming population

property children

List of child nodes

property nodeType

Type of the node (e.g., ADD, CONST, TERMINAL)

size(self: scheduler.ExprNode) → int

Get the size of the subtree rooted at this node

property terminalIndex

Index of the terminal if the node is a TERMINAL

class scheduler.FitnessEvaluator

Bases: pybind11_object

Evaluates a population of GP individuals by simulating a greedy schedule with each individual's expression tree as the priority function.

After evaluation every *Individual* in the population will have its `fitness` field set to a scalar value that reflects how well the tree minimised response time and energy consumption.

1.1 Fitness formula

Three raw objectives are collected for each individual:

- `compRatio` – fraction of available jobs that were successfully scheduled (higher is better).
- `meanResp` – average response time (completion - arrival) in minutes (lower is better).
- `energy` – total energy consumed across all satellites in watt- minutes (lower is better).

Each objective is normalised to `[0, 1]` across the current population so that differences in scale cannot dominate

the result. The final fitness is then:

$$\begin{aligned} f = & w \\ & \text{textjobs} \\ & \text{cdot} \\ & \text{widehat{r}} - w \\ & \text{texttime} \\ & \text{cdot} \\ & \text{widehat{t}} - w \\ & \text{textenergy} \\ & \text{cdot} \\ & \text{widehat{e}} \end{aligned}$$

where

hatcdot denotes population-normalised values and the default weights are (0.60, 0.25, 0.15).

Example

```
from scheduler import SchedulerGP, FitnessEvaluator, SimulationSnapshot
from scheduler import buildTerminalRegistry

gp = SchedulerGP(satellites, jobs)
gp.initialize(populationSize=100)

eval = FitnessEvaluator(buildTerminalRegistry())

for generation in range(50):
    snap = SimulationSnapshot(current_minute, satellites, jobs)
    eval.evaluate(gp.getPopulation(), snap) # writes fitness in place
    gp.solveNextGeneration()
```

evaluate (*self*: scheduler.FitnessEvaluator, *population*: scheduler.Population, *snap*: scheduler.SimulationSnapshot) → None

Score every individual in *population* and write the result into each individual's **fitness** field.

This is the only method the main GP loop needs to call each generation.

Parameters

- **population** (*list*[Individual]) – The current GP population, obtained via *SchedulerGP.getPopulation()*. Each individual's **fitness** field is updated in place.
- **snap** (SimulationSnapshot) – Read-only snapshot of the current simulation state. The evaluator runs a fresh greedy simulation for every individual, so **snap** is never modified.

Notes

Fitness values are population-relative: the same tree may receive a different score in a different generation if the rest of the population has changed.

evaluate_single (*self*: scheduler.FitnessEvaluator, *individual*: scheduler.Individual, *snap*: scheduler.SimulationSnapshot) → tuple

Evaluate a single individual and return its raw (un-normalised) objectives.

Useful for inspecting the best individual after evolution has finished.

Parameters

- **individual** (Individual) – The GP individual to evaluate.
- **snap** (SimulationSnapshot) – Simulation snapshot used for the evaluation.

Returns

(comp_ratio, mean_response_time, total_energy)

- **comp_ratio** – fraction of available jobs scheduled [0, 1].
- **mean_response_time** – average (completion - arrival) in minutes.
- **total_energy** – total energy consumed (watt-minutes).

Return type

tuple[float, float, float]

property weightEnergy

Weight applied to the energy-consumption penalty. Default 0.15. Increase this to make the GP prefer energy-efficient satellite assignments.

property weightJobs

Weight applied to the completion-ratio reward term. Default 0.60. Increase this to make the GP prioritise scheduling as many jobs as possible.

property weightTime

Weight applied to the mean-response-time penalty. Default 0.25. Increase this to make the GP prioritise faster job turnaround.

class scheduler.Individual

Bases: pybind11_object

Represents a single candidate solution in the Genetic Programming population

property fitness

Evaluated fitness score

property id

Unique identifier of the individual

property tree

The individual's expression tree root node

class scheduler.Job

Bases: pybind11_object

Represents a job/task to be scheduled

property arrivalTime**property disk****property executionServerName****property executionTime****property id****property imageSize**

property numberOfHops

property ram

property transferTime

property type

class scheduler.NodeType

Bases: pybind11_object

Enum representing the type of operation or value in an expression tree node

Members:

ADD : Addition operator

SUB : Subtraction operator

MUL : Multiplication operator

DIV : Division operator

MAX_OP : Maximum of two values

MIN_OP : Minimum of two values

NEG : Negation operator

ABS : Absolute value operator

INV : Inverse value operator

TERMINAL : Terminal node (variable)

CONST : Constant value node

ABS = <NodeType.ABS: 7>

ADD = <NodeType.ADD: 0>

CONST = <NodeType.CONST: 10>

DIV = <NodeType.DIV: 3>

INV = <NodeType.INV: 8>

MAX_OP = <NodeType.MAX_OP: 4>

MIN_OP = <NodeType.MIN_OP: 5>

MUL = <NodeType.MUL: 2>

NEG = <NodeType.NEG: 6>

SUB = <NodeType.SUB: 1>

TERMINAL = <NodeType.TERMINAL: 9>

NodeType.name -> str

property value

class scheduler.Population

Bases: pybind11_object

append(*self*: scheduler.Population, *x*: *GPIndividual*) → None

Add an item to the end of the list

clear(*self*: scheduler.Population) → None

Clear the contents

extend(*args, **kwargs)

Overloaded function.

1. extend(*self*: scheduler.Population, *L*: scheduler.Population) → None

Extend the list by appending all the items in the given list

2. extend(*self*: scheduler.Population, *L*: Iterable) → None

Extend the list by appending all the items in the given list

insert(*self*: scheduler.Population, *i*: int, *x*: *GPIndividual*) → None

Insert an item at a given position.

pop(*args, **kwargs)

Overloaded function.

1. pop(*self*: scheduler.Population) → *GPIndividual*

Remove and return the last item

2. pop(*self*: scheduler.Population, *i*: int) → *GPIndividual*Remove and return the item at index *i***class scheduler.Satellite**

Bases: pybind11_object

Represents a computing satellite resource

property bandwidth**property completedTasks****property cpuBusyUntil****property cpuCapacity****property deadTasks****property elevationAngle****property energyReserved****property id****property isAccessPoint****property latency****property name****property neighbors**

property networkBusyUntil

property networkCapacity

property orbitalSunset

property rejectedTasks

property remainingEnergy

property tasks

class scheduler.SchedulerGP

Bases: pybind11_object

Genetic Programming Scheduler Manager

getPopulation(*self*: scheduler.SchedulerGP) → *scheduler.Population*

Get the current population of individuals (mutable reference).

initialize(*self*: scheduler.SchedulerGP, *populationSize*: int = 200, *crossoverRate*: float = 0.8, *mutationRate*: float = 0.08, *elitismCount*: int = 2, *maxDepth*: int = 7, *hoistRate*: float = 0.05) → None

Initialize the GP population with parameters

printSchedule(*self*: scheduler.SchedulerGP) → None

Print the best schedule found so far

solveNextGeneration(*self*: scheduler.SchedulerGP) → None

Evaluate the current population and evolve to the next generation

class scheduler.SimulationSnapshot

Bases: pybind11_object

A self-contained, point-in-time snapshot of the simulation world.

The external application must construct and populate a snapshot before calling *FitnessEvaluator.evaluate()* for each GP generation. All satellite and job objects are copied into the snapshot, so subsequent changes to the originals have no effect on the evaluator.

Example

```
snap = scheduler.SimulationSnapshot(
    currentMinute = 0.0,
    satellites     = my_satellites,
    jobs          = my_jobs,
    uniformEnergy = 2.5,    # watts per minute per satellite
)
```

property currentMinute

Simulation clock at snapshot time (minutes from epoch).

property energyPerMinute

Energy consumed per minute of computation, indexed by satellite position in the *satellites* list. When non-empty this overrides *uniformEnergy*. Must have the same length as *satellites* if provided.

property jobs

Pool of jobs available for scheduling at this instant.

property satellites

Current state of every satellite in the constellation.

class scheduler.TerminalDef

Bases: pybind11_object

A named terminal variable used in GP expression trees (e.g. 'job.priority').

property name

Human-readable name of the terminal (e.g. 'job.priority').

scheduler.buildTerminalRegistry() → list[scheduler.TerminalDef]

Build and return the terminal registry used by the GP algorithm.

The registry maps human-readable names to field accessors that the expression tree evaluator uses to read job and satellite features at runtime. You must pass the returned list to *FitnessEvaluator* and, if you construct a *TreeBuilder* directly, to that as well.

Returns

Ordered list of terminal definitions. The index of each entry corresponds to the `terminalIndex` stored in `TERMINAL` nodes of the expression tree.

Return type

list[TerminalDef]

Example

```
from scheduler import buildTerminalRegistry, FitnessEvaluator

registry = buildTerminalRegistry()
evaluator = FitnessEvaluator(registry)
```


PYTHON MODULE INDEX

S

`scheduler, ??`

`schedulerlib, ??`